

Day 21: Weekly Revision

1. Day Number

Day 21

2. Topic Name

Revision of Days 15–20

3. Connection

During Days 15–20, you learned how to clean text and store information using Python collections.

Today's revision combines:

- String cleaning
- String slicing
- `replace()`
- Dictionaries
- Lists of dictionaries
- Sets

You will use these ideas to create and display a simple user profile.

4. Revision Summary of Days 15–20

Day 15: Clean Account Holder Name

You learned string methods such as:

- `strip()` — removes spaces from the beginning and end
- `title()` — formats each word with a capital first letter
- `lower()` — converts text to lowercase
- `upper()` — converts text to uppercase

Example:

```
name = "  rahul tiwari  "
clean_name = name.strip().title()
```

Result:

```
Rahul Tiwari
```

Day 16: Mask Account Number Using Slicing

You learned how to access part of a string using slicing.

```
account_number[-4:]
```

This returns the last four characters.

You also learned that `len()` can be used to find the number of characters in a string.

Day 17: Clean Phone Number

You learned how to remove unwanted characters using `replace()`.

```
phone.replace(" ", "")  
phone.replace("-", "")
```

For example:

```
98765-43210
```

Becomes:

```
9876543210
```

Day 18: Profile Dictionary

You learned that a dictionary stores information as **key-value pairs**.

```
profile = {  
    "name": "Rahul",  
    "city": "Bengaluru"  
}
```

Here:

- "name" is a key
 - "Rahul" is its value
 - "city" is another key
 - "Bengaluru" is its value
-

Day 19: List of Dictionaries

You learned how to store multiple records inside a list.

```
profiles = [  
    {"name": "Rahul", "city": "Bengaluru"},  
    {"name": "Amit", "city": "Delhi"}  
]
```

Each dictionary represents one record.

Day 20: Unique Cities Using Set

You learned that a set stores only unique values.

```
cities = {"Delhi", "Bengaluru", "Delhi"}
```

The duplicate "Delhi" is automatically removed.

Result:

```
{"Delhi", "Bengaluru"}
```

5. Important Topics

String methods

String methods help clean and format text.

Common methods:

```
strip()  
title()  
lower()  
upper()
```

Slicing

Slicing extracts a part of a string.

```
text[start:end]
```

Example:

```
phone[-4:]
```

This gives the last four characters.

replace()

replace() replaces one part of a string with another.

```
phone.replace("-", "")
```

An empty string "" means remove the character.

Dictionary

A dictionary stores related information using keys and values.

```
profile = {  
    "name": "Rahul",  
    "phone": "9876543210",  
    "city": "Bengaluru"  
}
```

List of dictionaries

A list of dictionaries can store multiple user records.

```
users = [  
    {"name": "Rahul"},  
    {"name": "Amit"}  
]
```

Set

A set stores unique values only.

```
cities = {"Delhi", "Mumbai", "Delhi"}
```

6. Foundational Notes

Strings are immutable

Python strings cannot be changed directly. String methods usually return a new string.

```
name.strip()
```

This does not permanently update name unless you save the result:

```
name = name.strip()
```

Method chaining

You can apply multiple string methods in one statement.

```
clean_name = name.strip().title()
```

Python first runs `strip()`, then runs `title()` on the cleaned result.

Multiple `replace()` calls

A phone number may contain both spaces and dashes.

You can clean both one after another:

```
clean_phone = phone.replace(" ", "").replace("-", "")
```

This is called **method chaining**.

Dictionary keys should be meaningful

Use clear keys such as:

```
"name"  
"phone"  
"city"
```

Avoid unclear keys such as:

```
"a"  
"b"  
"c"
```

Empty input is still a string

When the user presses Enter without typing anything, Python receives:

```
" "
```

You can check it using:

```
if city == "":
```

After using `strip()`, you can check:

```
if city.strip() == "":
```

This also detects input containing only spaces.

7. Easy Example

Suppose you want to clean a book title and store it in a dictionary.

```
book_title = " python basics "  
clean_title = book_title.strip().title()  
  
book = {  
    "title": clean_title,  
    "category": "Programming"  
}  
  
print(book)
```

Output:

```
{'title': 'Python Basics', 'category': 'Programming'}
```

This example uses:

- A string
 - `strip()`
 - `title()`
 - A dictionary
 - Key-value pairs
-

8. Revision Problem Statement

Create a Python program that:

1. Asks the user to enter their name.

2. Asks the user to enter their phone number.
3. Asks the user to enter their city.
4. Cleans the name by:
 - Removing extra spaces from the beginning and end
 - Formatting it using title case
5. Cleans the phone number by:
 - Removing spaces
 - Removing dashes
6. Stores the cleaned name, cleaned phone number, and city in a profile dictionary.
7. Prints the completed profile.

Example input:

```
Enter your name:  rahul tiwari
Enter your phone: 98765-43210
Enter your city: Bengaluru
```

Expected profile information:

```
Name: Rahul Tiwari
Phone: 9876543210
City: Bengaluru
```

You should create the logic yourself rather than copying a complete solution.

9. Concepts Used

This problem uses:

- Variables
- `input()`
- Strings
- `strip()`
- `title()`
- `replace()`
- Functions
- Parameters
- Return values
- Dictionaries
- Dictionary keys and values
- Basic empty-input validation
- `print()`

A set and list of dictionaries are part of this week's revision, but they are not required in the main profile problem.

10. Thought Process

Step 1: Identify the inputs

The program needs three pieces of information:

- Name

- Phone number
- City

Store each input in a separate variable.

Step 2: Clean the name

The name might contain:

- Spaces before the name
- Spaces after the name
- Lowercase letters
- Uppercase letters

Use string methods to remove outside spaces and format the name.

For example:

```
" rAHUL tiWARI "
```

Should become:

```
"Rahul Tiwari"
```

Step 3: Clean the phone number

The phone number might contain:

- Spaces
- Dashes

For example:

```
"98765-43210"
```

Should become:

```
"9876543210"
```

The phone number should remain a string because you are cleaning characters rather than doing calculations with it.

Step 4: Handle the city

Remove unnecessary spaces from the beginning and end.

Decide what should happen when the city is empty. For this beginner problem, you could store a default value such as:

```
Not provided
```

Step 5: Create the dictionary

Create a dictionary with three keys:

```
name
phone
city
```

Store the cleaned values against these keys.

Step 6: Print the profile

You may:

- Print the complete dictionary, or
- Print each dictionary value separately using its key

Printing each value separately is usually easier to read.

11. Pseudocode

```
START

DEFINE a function to clean the name
    Remove spaces from the beginning and end
    Convert the name to title case
    RETURN cleaned name

DEFINE a function to clean the phone number
    Remove spaces
    Remove dashes
    RETURN cleaned phone number

DEFINE a function to create the profile
    Receive name, phone, and city
    Create a dictionary
    Store name using the "name" key
    Store phone using the "phone" key
    Store city using the "city" key
    RETURN the dictionary

ASK the user for name
ASK the user for phone number
ASK the user for city

CALL the name-cleaning function
CALL the phone-cleaning function
REMOVE extra outside spaces from the city

IF the city is empty
    Use "Not provided" as the city

CALL the profile-creation function

PRINT the profile information

END
```

12. Suggested Solving Approach: Functional Approach

Divide the program into small functions.

Function 1: Clean the name

Possible responsibility:

```
clean_name(name)
```

It should:

- Receive the original name
- Remove outside spaces
- Format the name
- Return the cleaned name

Function 2: Clean the phone

Possible responsibility:

```
clean_phone(phone)
```

It should:

- Receive the original phone number
- Remove spaces
- Remove dashes
- Return the cleaned phone number

Function 3: Create the profile

Possible responsibility:

```
create_profile(name, phone, city)
```

It should:

- Receive the three cleaned values
- Store them in a dictionary
- Return the profile dictionary

Function 4: Display the profile

Optional responsibility:

```
show_profile(profile)
```

It should:

- Receive the dictionary
- Access its values using keys
- Print the profile clearly

The program flow becomes:

Input → Clean values → Create dictionary → Display profile

13. Easy Edge Cases

Extra spaces in the name

Input:

" neha sharma "

Expected cleaned name:

"Neha Sharma"

Use `strip()` before formatting the name.

Phone number with dashes

Input:

"98765-43210"

Expected cleaned phone:

"9876543210"

Use `replace()` to remove `-`.

Phone number with spaces and dashes

Input:

"98765 - 43210"

Expected cleaned phone:

"9876543210"

Remove both spaces and dashes.

Empty city

Input:

" "

Possible stored value:

"Not provided"

Remember to check the city after applying `strip()`.

Name containing only spaces

Input:

```
"    "
```

After `strip()`, it becomes an empty string.

You may store a default value such as:

```
"Unknown"
```

14. Common Mistakes to Avoid

Forgetting to save the cleaned result

Incorrect idea:

```
name.strip()
name.title()
```

The returned values are not saved.

The cleaned result should be assigned to a variable.

Using `int()` for a phone number

A phone number should normally be stored as a string.

Converting it to an integer may:

- Remove a starting zero
- Fail when spaces or dashes are present
- Make string cleaning harder

Removing dashes but not spaces

Cleaning only `" - "` will not remove spaces.

Think about all unwanted characters in the input.

Creating the dictionary before cleaning the values

Clean the values first, then store them in the dictionary.

Better flow:

```
Raw input → Clean input → Store cleaned input
```

Using inconsistent dictionary keys

For example:

```
"profile_name"
"Phone"
"cityName"
```

Use simple and consistent keys:

```
"name"  
"phone"  
"city"
```

Checking for an empty city before `strip()`

An input containing spaces is not equal to "" until it is cleaned.

```
"  "
```

After `strip()`:

```
" "
```

15. Quick Self-Check Questions

1. Which string method removes spaces from the beginning and end?
 2. Which method can remove dashes from a phone number?
 3. Why is a phone number usually stored as a string instead of an integer?
 4. How do you access the value stored under the "city" key in a dictionary?
 5. What is the main difference between a list and a set regarding duplicate values?
-

16. Hint Only

Create separate functions for cleaning the name and phone number.

Think about this program flow:

```
name input  
  ↓  
strip and title  
  ↓  
cleaned name  
  
phone input  
  ↓  
remove spaces and dashes  
  ↓  
cleaned phone  
  
cleaned values  
  ↓  
profile dictionary  
  ↓  
print using dictionary keys
```

Useful methods and structures:

```
.strip()  
.title()  
.replace()  
{"key": value}
```

Try to complete the program using at least three functions:

```
clean_name()  
clean_phone()  
create_profile()
```